

API AUDIENCIA VIRTUAL

Documentacao tecnica simplificada para apresentacao em entrevista

Projeto desenvolvido em Java com Spring Boot para organizar usuarios, partes, agendas e audiencias virtuais.

Informacoes do projeto	Descricao
Tipo	API REST
Versao da documentacao	1.0
Porta configurada	8081
Documentacao interativa	Swagger UI

Como apresentar em uma frase

Esta e uma API REST organizada em camadas, usando Spring Boot, JPA e Hibernate para persistir dados de usuarios, partes, agendas e audiencias virtuais, com validacao e documentacao automatica pelo Swagger.

1. Objetivo do sistema

A API Audiencia Virtual representa o fluxo basico de um sistema judicial: uma pessoa pode ser cadastrada como usuario, uma parte possui dados do processo, uma agenda e associada a parte e uma audiencia virtual e criada a partir dessa agenda.

O projeto foi estruturado para separar responsabilidades. Assim, cada parte do codigo possui uma funcao clara e pode ser alterada com menor risco de afetar as demais.

Fluxo principal:

- Cadastrar ou consultar usuarios.
- Cadastrar partes vinculadas a um usuario.
- Criar uma agenda para uma parte.
- Agendar uma audiencia virtual para uma agenda.
- Consultar, pesquisar e excluir registros.

2. Arquitetura em camadas

A aplicacao segue uma arquitetura simples em camadas, muito comum em APIs Java.

Camada	Responsabilidade	Exemplo
Controller	Recebe requisicoes HTTP, valida entradas e devolve respostas.	UsuarioController, AgendaController
Service	Concentra regras de negocio e coordena operacoes.	AudienciaVirtualService
Repository	Acessa o banco por meio do Spring Data JPA.	UsuarioRepository
Model / Entity	Representa dados e tabelas do banco.	Usuario, Parte, Agenda
DTO	Define o formato de entrada e saida.	AudienciaVirtualRequestDto
Configuration	Configura recursos transversais.	OpenApiConfig

Como explicar na entrevista

O Controller nao acessa o banco diretamente. Ele chama o Service, que aplica as regras de negocio. O Service usa o Repository, e o Repository conversa com o banco. Essa separacao facilita testes, manutencao e evolucao do sistema.

3. Classes de dominio (Model)

As classes do pacote Model sao entidades JPA. Cada entidade possui uma representacao no banco por meio da anotacao @Entity.

Classe	O que representa	Relacionamento ou detalhe
Usuario	Pessoa cadastrada no sistema.	Possui id, nome, CPF e funcao. CPF obrigatorio, unico e validado.
Parte	Parte de um processo judicial.	Herda de Usuario e acrescenta numeroProcesso. Usa heranca JPA JOINED.
Agenda	Agenda relacionada a uma parte.	Registros de agenda apontam para uma parte.
AudienciaVirtual	Audiencia virtual agendada.	Possui relacao um-para-um com Agenda.

Validacoes importantes da classe Usuario:

- @NotBlank impede nome, CPF ou funcao vazios.
- @Pattern exige CPF com 11 numeros.
- O setter do CPF remove pontos, hifens e outros caracteres nao numericos.
- O CPF e unico no banco e o Controller responde 409 quando ja existe.

Exemplo de entrada aceita:

```
{
  "nome": "Maria da Silva",
  "cpf": "123.456.789-00",
  "funcao": "Advogada"
}
```

4. Controllers e endpoints

Os Controllers são responsáveis por expor os recursos da API por meio de URLs e métodos HTTP.

Recurso	Endpoints principais	Objetivo
Usuarios	GET /v1/usuarios GET /v1/usuarios/{id} POST /v1/usuarios DELETE /v1/usuarios/{id}	Gerenciar usuarios.
Partes	GET /v1/partes GET /v1/partes/{id} POST /v1/partes DELETE /v1/partes/{id}	Gerenciar partes processuais.
Agendas	GET /v1/agendas GET /v1/agendas/{id} POST /v1/agendas DELETE /v1/agendas/{id}	Gerenciar agendas de partes.
Audiencias	POST /v1/audiencias-virtuais GET /v1/audiencias-virtuais GET /v1/audiencias-virtuais/{id} GET /v1/audiencias-virtuais/buscar?nome=... DELETE /v1/audiencias-virtuais/{id}	Criar e consultar audiencias.

Exemplo de criação de audiência:

```
POST /v1/audiencias-virtuais
Content-Type: application/json
```

```
{
  "agendaId": 1
}
```

A resposta usa `AudienciaVirtualResponseDto` para retornar dados úteis, como id da audiência, id da agenda, id da parte e nome da parte.

5. Services e regras de negocio

Os Services evitam que as regras fiquem espalhadas nos Controllers.

Service	Principais responsabilidades
UsuarioService	Lista, consulta, salva, verifica CPF duplicado e exclui usuarios.
ParteService	Lista, consulta, salva e exclui partes.
AgendaService	Lista, consulta, salva e exclui agendas.
AudienciaVirtualService	Valida agenda, confirma a parte, impede duas audiencias para a mesma agenda, cria, consulta, pesquisa e exclui audiencias.

Regra mais importante do agendamento

Antes de criar uma audiencia, o sistema busca a agenda pelo ID. Depois confirma se a agenda possui uma parte valida e verifica se ja existe uma audiencia para aquela agenda. Isso evita referencias invalidas e duplicidade de agendamento.

A anotacao @Transactional garante que a operacao de negocio seja tratada como uma unidade: se houver erro durante o processo, a transacao pode ser revertida.

6. Repositories e banco de dados

Os Repositories usam Spring Data JPA. Ao estender JpaRepository, as operacoes basicas de CRUD sao disponibilizadas sem a necessidade de escrever SQL manualmente.

Repository	Entidade	Consulta especial
UsuarioRepository	Usuario	existsByCpf verifica CPF duplicado.
ParteRepository	Parte	Usa operacoes CRUD padrao.
AgendaRepository	Agenda	Usa operacoes CRUD padrao.
AudienciaVirtualRepository	AudienciaVirtual	Verifica audiencia por agenda e pesquisa pelo nome da parte.

Configuracao atual:

- H2 em memoria: util para desenvolvimento e demonstracao.
- spring.jpa.hibernate.ddl-auto=create-drop: tabelas criadas ao iniciar e removidas ao encerrar.
- MySQL Connector declarado para permitir troca futura para MySQL.
- ApiApplication possui CommandLineRunner que carrega dados iniciais de exemplo.

Tecnologia de persistencia

H2 + JPA + Hibernate: o Java trabalha com objetos, o JPA define o mapeamento e o Hibernate executa a comunicacao com o banco.

7. Tecnologias utilizadas

Tecnologia	Uso no projeto	Como explicar
Java 17	Linguagem da aplicacao.	Base da API e da orientacao a objetos.
Spring Boot 4.1.1	Inicializacao e configuracao.	Reduz configuracao manual e facilita APIs.
Spring MVC	Criacao dos endpoints REST.	Mapeia metodos Java para GET, POST e DELETE.
Spring Data JPA	Acesso ao banco.	Oferece repositories e operacoes CRUD.
Hibernate	Implementacao do JPA.	Converte objetos Java em tabelas e SQL.
H2 Database	Banco em memoria.	Permite executar sem instalar banco externo.
MySQL Connector	Compatibilidade com MySQL.	Facilita usar MySQL em outro ambiente.
Jakarta Validation	Validacao de entradas.	Garante campos obrigatorios e CPF correto.
Springdoc OpenAPI	Swagger UI e documentacao.	Permite testar endpoints pelo navegador.
Maven	Dependencias e build.	Baixa bibliotecas e executa testes.
JUnit / Spring Boot Test	Estrutura de testes.	Permite validar o contexto da aplicacao.
Lombok	Dependencia disponivel.	Pode reduzir getters, setters e construtores.

8. Swagger e como demonstrar

A documentacao OpenAPI foi integrada com Springdoc. Depois de iniciar a API, a interface interativa pode ser acessada no navegador:

`http://localhost:8081/swagger-ui.html`

Tambem existe o documento OpenAPI em formato JSON:

`http://localhost:8081/v3/api-docs`

No Swagger, e possivel expandir um endpoint, clicar em Try it out, informar os dados e executar a chamada sem abrir o Postman.

Sequencia sugerida para a apresentacao:

- Consultar GET /v1/partes e mostrar os dados iniciais.
- Criar um usuario com POST /v1/usuarios.
- Criar ou consultar uma agenda.
- Criar uma audiencia com POST /v1/audiencias-virtuais.
- Pesquisar audiencias pelo nome da parte.

9. Respostas HTTP e tratamento basico

Status	Significado no projeto
200 OK	Consulta realizada com sucesso.
201 Created	Registro criado com sucesso.
204 No Content	Registro excluido com sucesso.
400 Bad Request	Dados invalidos, como campos obrigatorios ausentes.
404 Not Found	Recurso nao encontrado pelo ID informado.
409 Conflict	CPF ja cadastrado ou conflito de dados.

10. Pontos fortes para mencionar na entrevista

- Separacao clara entre Controller, Service, Repository e Model.
- Uso de DTO para controlar dados retornados no agendamento.
- Validacao de dados com mensagens em portugues.
- Regra para impedir audiencia duplicada na mesma agenda.
- Documentacao interativa via Swagger.
- Banco H2 para facilitar execucao e demonstracao local.
- Possibilidade de substituir H2 por MySQL usando o conector declarado.

Resposta pronta para “qual foi seu papel no projeto?”

Eu organizei a API em camadas, modeliei as entidades e seus relacionamentos, criei os endpoints REST, implementei as regras de agendamento, adicionei validacoes para evitar dados invalidos e documentei os endpoints com Swagger para facilitar testes e integracao.

11. Resumo final

A API Audiencia Virtual e uma aplicacao REST Java que demonstra conceitos importantes de desenvolvimento backend: modelagem de entidades, heranca e relacionamentos JPA, operacoes CRUD, validacao, regras de negocio, documentacao OpenAPI e organizacao em camadas.

O ponto principal para a entrevista e mostrar que cada tecnologia foi usada com uma finalidade: Spring Boot para estruturar a aplicacao, Spring MVC para expor a API, JPA e Hibernate para persistir dados, Validation para proteger as entradas e Swagger para documentar e testar os endpoints.

Checklist para a apresentacao

- Explicar o objetivo do sistema em 30 segundos.
- Mostrar a arquitetura em camadas.
- Explicar o relacionamento Usuario -> Parte -> Agenda -> AudienciaVirtual.
- Executar um POST e um GET no Swagger.
- Comentar validacoes e prevencao de duplicidade.
- Finalizar destacando facilidade de manutencao e evolucao.

Documento gerado com base na implementacao atual do projeto.